

Actividad 1: "Del Cuello de Botella a la Concurrency Multihilo"

PROGRAMACION DE APLICACIONES DE RED

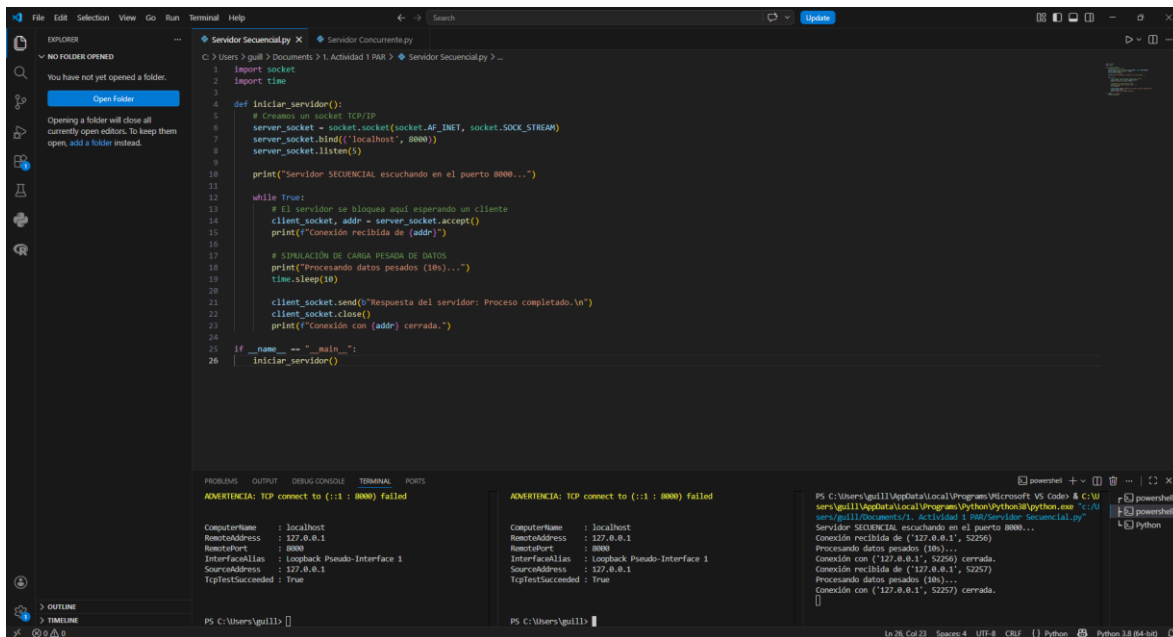
Luis Guillermo Mejenes Chacon, Javier Rodríguez Rodríguez y Jorge
Enrique Soberanes Espinosa
INGENIERIA EN DATOS E INTELIGENCIA ORGANIZACIONAL

1. Objetivo General

Analizar y demostrar de forma práctica la diferencia de rendimiento entre un servidor TCP secuencial (iterativo) y un servidor concurrente (multihilo), identificando los cuellos de botella y aplicando soluciones de escalabilidad.

2. Fase 1: Análisis del Servidor Secuencial

En el modelo secuencial, el servidor atiende una sola conexión a la vez. Mientras el proceso de atención (simulado con un retardo de 10 segundos) está activo, el servidor es incapaz de responder a nuevas peticiones.



The screenshot shows a VS Code editor with a Python file named 'Servidor Secuencial.py'. The code defines a function 'iniciar_servidor()' that creates a TCP socket, binds it to 'localhost' on port 8000, and enters a loop where it accepts connections, prints the client address, simulates processing with 'time.sleep(10)', and sends a response. The main block calls this function. The terminal output shows a warning for a failed connection to port 8000, followed by successful connections from '127.0.0.1' on ports 52256 and 52257, each followed by a 10-second delay and a response.

```
1 import socket
2 import time
3
4 def iniciar_servidor():
5     # Creamos un socket TCP/IP
6     server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
7     server_socket.bind(('localhost', 8000))
8     server_socket.listen(5)
9
10    print("Servidor SECUENCIAL escuchando en el puerto 8000...")
11
12    while True:
13        # El servidor se bloquea aquí esperando a un cliente
14        client_socket, addr = server_socket.accept()
15        print(f"Conexión recibida de {addr}")
16
17        # Simulación de carga pesada de datos
18        print(f"Procesando datos pesados (10s)...")
19        time.sleep(10)
20
21        client_socket.send(f"Respuesta del servidor: Proceso completado.\n")
22        client_socket.close()
23        print(f"Conexión con {addr} cerrada.")
24
25 if __name__ == "__main__":
26     iniciar_servidor()
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

ADVERTENCIA: TCP connect to (:::8000) failed

ADVERTENCIA: TCP connect to (:::8000) failed

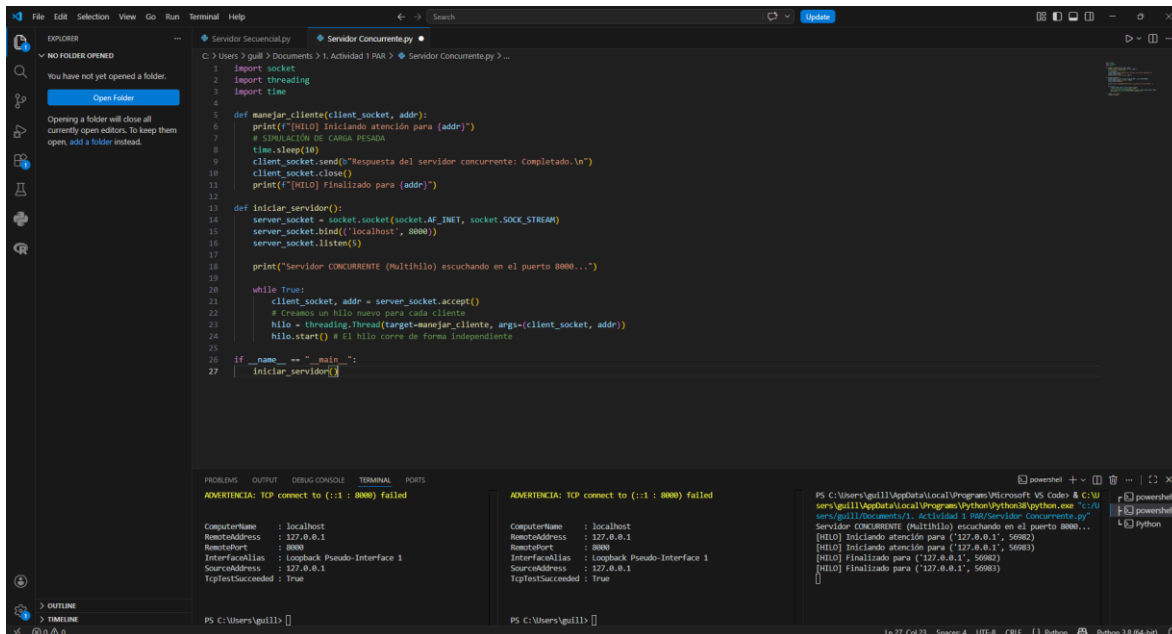
PS C:\Users\guill\AppData\Local\Programs\Microsoft VS Code & C:\W...
serv\guill\Aplicaciones\Programas\Python\Python38\python.exe C:\W...
serv\guill\documentos\1_Actividad 1 PAR\Servidor Secuencial.py
Servidor SECUENCIAL escuchando en el puerto 8000...
Conexión recibida de ('127.0.0.1', 52256)
Procesando datos pesados (10s)...
Conexión con ('127.0.0.1', 52256) cerrada.
Conexión recibida de ('127.0.0.1', 52257)
Procesando datos pesados (10s)...
Conexión con ('127.0.0.1', 52257) cerrada.

Análisis: Como se observa en la evidencia, el segundo cliente (con puerto efímero distinto) se conecta exitosamente a nivel de transporte (TCP handshake), pero la aplicación no lo procesa hasta que el primer bucle finaliza. Esto se debe a que el flujo de ejecución está detenido en la instrucción `time.sleep()`.

3. Fase 2: Implementación de Servidor Concurrente

Utilizando la librería `threading`, se modificó el servidor para que cada conexión entrante sea gestionada en un hilo independiente, permitiendo que el hilo principal retorne

inmediatamente a escuchar nuevas conexiones.



The screenshot shows a VS Code editor with a file named 'Servidor Concurrente.py'. The code defines a function 'manejar_cliente' that simulates a heavy load by sleeping for 10 seconds before sending a response. It then defines 'iniciar_servidor' which uses a 'while True' loop to accept connections and spawn new threads for each client using 'threading.Thread'. The main block starts the server. The terminal output shows a warning about a failed TCP connection, followed by logs indicating the server is listening on port 8080 and successfully processing two concurrent requests from 127.0.0.1, each taking approximately 10 seconds.

```
1 import socket
2 import threading
3 import time
4
5 def manejar_cliente(client_socket, addr):
6     print(f"[Hilo] Iniciando atención para {addr}")
7     # SIMULACIÓN DE CARGA PESADA
8     time.sleep(10)
9     client_socket.send(b"Respuesta del servidor concurrente: Completado.\n")
10    client_socket.close()
11    print(f"[Hilo] Finalizado para {addr}")
12
13 def iniciar_servidor():
14     server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
15     server_socket.bind(('localhost', 8080))
16     server_socket.listen(5)
17
18     print("Servidor CONCURRENTE (Multihilo) escuchando en el puerto 8080...")
19
20     while True:
21         client_socket, addr = server_socket.accept()
22         # Creamos un hilo nuevo para cada cliente
23         hilo = threading.Thread(target=manejar_cliente, args=(client_socket, addr))
24         hilo.start() # El hilo corre de forma independiente
25
26 if __name__ == "__main__":
27     iniciar_servidor()
```

PROBLEMAS OUTPUT DEBUG CONSOLE TERMINAL PORTS

ADVERTENCIA: TCP connect to (::1 : 8080) failed

ADVERTENCIA: TCP connect to (::1 : 8080) failed

PS C:\Users\guill\AppData\Local\Programs\Microsoft VS Code\ & C:\W...
serv\guill\Documents\1_Actividad 1 PAR\Servidor Concurrente.py
Servidor CONCURRENTE (Multihilo) escuchando en el puerto 8080...
[Hilo] Iniciando atención para ("127.0.0.1", 56962)
[Hilo] Iniciando atención para ("127.0.0.1", 56963)
[Hilo] Finalizado para ("127.0.0.1", 56962)
[Hilo] Finalizado para ("127.0.0.1", 56963)

ComputerName : localhost
RemoteAddress : 127.0.0.1
RemotePort : 8080
InterfaceName : Loopback Pseudo-Interface 1
SourceAddress : 127.0.0.1
TestSucceeded : True

ComputerName : localhost
RemoteAddress : 127.0.0.1
RemotePort : 8080
InterfaceName : Loopback Pseudo-Interface 1
SourceAddress : 127.0.0.1
TestSucceeded : True

PS C:\Users\guill>

Análisis: La implementación multihilo permite procesar N solicitudes en el tiempo que le tomaría a una sola (limitado solo por recursos de hardware). En la prueba, ambos clientes finalizaron su proceso casi al mismo tiempo, reduciendo el tiempo de espera total de 20s a 10s aproximadamente.

4. Fase 3: Reflexión sobre Concurrency Crítica

Si 100 clientes paralelos intentaran modificar una variable global simultáneamente (ej. Ventas = Ventas + 1), ocurriría una Condición de Carrera. El valor final sería impredecible porque la operación de incremento no es atómica a nivel de CPU.

Para evitarlo se implementaría una Herramienta de Exclusión Mutua: Se debe utilizar un Lock (Semáforo binario).

En Python, se implementa así:

```
mutex = threading.Lock()
mutex.acquire() # Entrada a sección crítica
ventas += 1
mutex.release() # Salida
```

5. Conclusiones por Integrante

INTEGRANTE	FASE 1: SECUENCIAL	FASE 2: CONCURRENTE
Luis Guillermo Mejenes Chacon	Pude observar cómo un solo proceso pesado puede detener toda la operatividad de un servicio haciendo todo más lento.	La delegación de tareas a hilos es la base fundamental para cualquier servidor web moderno ayudando así a la eficiencia del servidor.
Javier Rodríguez Rodríguez	Es impresionante cómo el sistema operativo mantiene una cola de espera, aunque la aplicación esté bloqueada.	El manejo de hilos optimiza el tiempo de respuesta total del sistema de forma drástica mejorando todo.
Jorge Enrique Soberanes Espinosa	El modelo secuencial es útil solo para tareas extremadamente simples o servidores de baja demanda porque si no se un cuello de botella enorme.	Aprendí que la concurrencia es poderosa, pero requiere cuidado con el acceso a datos compartidos y así.

GitHub

<https://github.com/JaceBeleren11/Actividad-1-Del-Cuello-de-Botella-a-la-Concurrencia-Multihilo>

Bibliografía

Tanenbaum, A. S., & Van Steen, M. (2007). Sistemas Distribuidos. Principios y Paradigmas, 2da Edición.

Capítulo 3, Sección 3.1.2 ("Hilos en sistemas distribuidos").

Kurose, J. F., & Ross, K. W. (2017). Redes de Computadoras, 7ma Edición. Capítulo 3, Sección 3.2 ("Multiplexación y Demultiplexación").